Event & Program Tracker

System Architecture & Workflow Reference

How Users, Organisers and Admins flow through the system, how the MySQL database stores and relates data, what happens during registration and login sessions, what data persists, and what has been built so far.

Bachelor of Computer Science Semester Project | Spring Boot, Thymeleaf, Bootstrap 5, MySQL, Spring Security

Generated 12 June 2026 | Branch: frontend-ui

---

1. The big picture

Event & Program Tracker is a web application where people discover and sign up for events, organisers publish and manage those events, and an administrator oversees the whole platform. It is built on the classic Spring Boot layered architecture:

- **Browser (Thymeleaf + Bootstrap 5)** — the pages the user sees and clicks.
- **Controllers** — receive each web request (e.g. `/dashboard`, `/events/5/register`) and decide which page to return.
- **Services** — the business rules (e.g. prevent duplicate registration, count registrations).
- **Repositories (Spring Data JPA)** — translate Java method calls into SQL automatically.
- **MySQL database** — the permanent store on disk where every user, event and registration lives.

A request flows **down** through these layers and the answer flows back **up**: Browser to Controller to Service to Repository to MySQL, and back again as a rendered HTML page.

---

2. The three roles — objective & functions

Every account has exactly one **role**, stored on the user record as a fixed value: `USER`, `ORGANIZER`, or `ADMIN`. The role decides which pages the account is allowed to open and what it can do.

2.1 USER (Participant)

Objective: discover events and register to attend them.

- Browse the public event list and open any event's details.
- Register for an event, and cancel that registration later.
- View My Registrations — the events they have signed up for.
- See a personal dashboard with their registration count and upcoming events.
- Manage their own profile.

2.2 ORGANIZER

Objective: create and run events, and see who registered.

- Create a new event, edit it, and delete it.
- View My Events — only the events they themselves own.
- See the registration list for their events (scoped to their events only).
- View simple analytics — a fill-rate bar (registrations vs. capacity) per event.
- Has a dashboard showing total events, total registrations across their events, and upcoming events.

2.3 ADMIN

Objective: oversee and moderate the entire platform.

- Manage all platform users (view everyone, delete an account).
- View and manage every event on the platform (edit / delete any event).
- View platform-wide reports: total users, total events, total registrations.
- Has a dashboard aggregating every account, event and registration in the system.

Note: the project brief mentions an organiser approval workflow (pending to approved). The buttons exist visually, but there is currently no `status` field on the user record, so approval is not yet functional — see section 8 (limitations).

---

3. How each role flows through the system

Login is shared by all three roles. After login, Spring Security knows the role and `/dashboard` automatically forwards each person to the correct dashboard.

3.1 Shared entry flow

Visitor → **/auth/register** (create account) → **/auth/login** (sign in) → **/dashboard** → routed by role to the User, Organiser, or Admin dashboard.

3.2 USER flow

Login → User Dashboard → **Browse Events** → Open an event → **Register** → appears in **My Registrations** → (optional) **Cancel** → row removed.

3.3 ORGANIZER flow

Login → Organiser Dashboard → **Create Event** (form) → event saved and owned by them → appears in **My Events** → **Edit / Delete** → view **Registrations** and **Analytics** for their events.

3.4 ADMIN flow

Login → Admin Dashboard → **Manage Users** (view / delete), **All Events** (edit / delete any), **Reports** (live platform totals).

4. What database we use and how it works

The application uses a **MySQL** relational database called `event_tracker_db`, running on the standard port 3306. The connection is configured in `application.properties`:

```
spring.datasource.url = jdbc:mysql://localhost:3306/event_tracker_db
spring.datasource.username = root
spring.jpa.hibernate.ddl-auto = update
```

How Java talks to the database (JPA / Hibernate)

We never write raw SQL by hand. Each table is represented by a Java class called an **entity** (`User`, `Event`, `Registration`). Spring Data JPA + Hibernate sit in the middle and:

- **Create the tables for us.** Because `ddl-auto=update` is set, Hibernate reads the entity classes on startup and automatically creates or updates the matching tables. We do not run manual CREATE TABLE scripts.
- **Generate the SQL for us.** A repository method like `findByEmail(email)` is turned into a real `SELECT * FROM users WHERE email = ?` behind the scenes.
- **Map rows to objects.** A row in the `users` table becomes a `User` Java object, and vice-versa.

The tables

Three real tables exist in the current data model:

Table	Holds	Key columns
users	Every account (all roles)	id, name, email (unique), password (hashed), role, phone, profile_picture, created_at
events	Every event	id, title, description, location, event_date, event_time, capacity, category, status, price, organizer_id (FK), image_url, created_at
registrations	Each link of user signed up for event	id, user_id (FK), event_id (FK), registration_date, status, with a UNIQUE(user_id, event_id) rule

The brief also lists separate `roles`, `categories` and `feedback` tables. In the current build these are simplified: **role** and **category** are stored directly as text columns rather than separate tables, and **feedback** is not yet implemented.

---

5. The relationships between data

The three tables connect through **foreign keys** — a column in one table that points at the primary key (id) of another. This is what makes it a relational database.

Relationship	Meaning	How it is wired
One User to Many Registrations	A person can sign up for many events	registrations.user_id to users.id
One Event to Many Registrations	An event can have many attendees	registrations.event_id to events.id
One Organiser to Many Events	An organiser owns many events	events.organizer_id to users.id

The `registrations` table is a **join (bridge) table**: it sits between users and events to express the many-to-many relationship users attend events. Its `UNIQUE(user_id, event_id)` constraint is what physically prevents the same person registering for the same event twice — the database itself rejects a duplicate.

Simple ERD (text form):

```
USERS ----< (organizer_id) ---- EVENTS
  |                            |
  | (user_id)     (event_id)
  +------< REGISTRATIONS >------+
```

6. What happens when you register — and is it tied to a session?

It helps to separate two different meanings of the word register:

6.1 Account registration (sign-up)

On `/auth/register` the form is submitted, the service checks the email is not already taken, the password is **encrypted with BCrypt** (never stored as plain text), and a new row is inserted into the `users` table. The person is then redirected to login.

6.2 Event registration (signing up to attend)

When a logged-in user clicks **Register** on an event, the service first checks whether a registration already exists for that user + event pair. If not, it inserts a new row into the `registrations` table with status `REGISTERED` and a timestamp. Cancelling deletes that row.

6.3 Is it integrated with a session?

Yes — but the session and the registration are two different things, and it is worth being precise:

- **The login session is temporary (in memory).** When you log in, Spring Security creates an HTTP session identified by a `JSESSIONID` cookie in your browser. That session remembers who you are and your role so you don't have to log in on every click. It lives in the server's memory and disappears when you log out, the cookie expires, or the server restarts.
- **The registration itself is permanent (on disk).** The session is only used to know which user is clicking. The actual registration is written as a real row in MySQL. So even after you log out and the session is gone, your registration still exists — you'll see it again next time you log in.

In short: the **session** identifies you for the duration of your visit; the **registration** is saved into the database and outlives the session.

---

7. Is the data volatile, or does it remain?

Both kinds exist in the system, and it is important not to confuse them.

Kind of data	Where it lives	Volatile or permanent?
Users, events, registrations	MySQL database (on disk)	**Permanent** — survives restarts, logout, and shutdown.
Login session / who is logged in	Server memory + JSESSIONID cookie	**Volatile** — lost on logout, cookie expiry, or restart.
Flash messages (e.g. Account created)	Session, for one redirect	**Volatile** — shown once, then gone.
Seeded demo accounts and events	Inserted by DataInitializer on first run	**Permanent** once created (not re-seeded if data already exists).

What data remains? Everything stored in the three MySQL tables — every account, every event, and every registration — remains permanently, because `ddl-auto=update` only updates the table structure; it never wipes your data. The only things that do *not* remain are the in-memory login session and one-time flash messages.

One caveat: if a teammate ever sets `ddl-auto` to `create` or `create-drop` instead of `update`, Hibernate would drop and rebuild the tables on every start and the data would be lost. The current setting (`update`) is the safe, data-preserving one.

---

8. What has been built so far

The project is past the frontend-only stage. It now has full entities, repositories, services, Spring Security, and a database seeder — and the role pages are connected to live data. In terms of the 14-phase plan this is **Phase 13 (Backend Integration)** moving into **Phase 14 (Demo Readiness)**.

Working end-to-end

Area	Status
Register / Login / Logout (BCrypt + Spring Security)	**Working**
Public event list — live events from the database	**Working**
Event details, Create, Edit, Delete	**Working** (edit and delete bugs fixed)
Register / Cancel for an event	**Working**
User: My Registrations (live, with Cancel)	**Working**
Organiser: My Events, Registrations, Analytics (scoped)	**Working**
Admin: Manage Users, All Events, Reports (live counts)	**Working**
All three dashboards — live stats and lists	**Working**

Known limitations (by design / not yet built)

- **Saved Events** — static placeholder; no bookmark entity in the data model yet.
- **Organiser approval** (pending to approved) — buttons are visual only; no status field on the user yet.

- **Category filter chips** filter on the page only; server-side search works by title.
- **Feedback / ratings** — table from the brief is not implemented yet.
- A few decorative widgets (page views, ratings, uptime) are static with no backing data.

Seed accounts for testing

Role	Email	Password
Admin	admin@eventtracker.com	admin123
Organiser	organizer@eventtracker.com	org123
User	user@eventtracker.com	user123

Run with `./mvnw spring-boot:run` (make sure MySQL is running), then open `http://localhost:8080`.